Predictive and Preference-Aware Compiler Optimization Framework

Presented By:
Satyam Singh Kumre (206124027)

Guided By:
Dr. Priyanka Panigrahi

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI

2. PROBLEM STATEMENT, OBJECTIVES & MOTIVATION

Problem Statement:
Standard compiler optimizations (e.g., Clang/GCC -O3, -Oz) use rigid, one-size-fits-all heuristic pass sequences, failing to adapt to specialized source code architecture. Lack of preference awareness makes it hard to strike a precise balance between Execution Speed, Code Size, and Compile Time.

Objectives:
To design and build a Machine Learning-based Predictive Compiler Optimization Framework on LLVM/Clang that dynamically selects optimal pass sequences per program and introduces a 3D Metric Priority Vector (Speed, Size, Compile-Time) for developer-steered output.

Motivation:

- Replace rigid -O3 heuristics with per-program ML prediction, delivering measurable real-world speedups.
- Enable developer control over compiler output — a first in ML-compiler research.
- Ultra-low 1.25 ms inference overhead: suitable for production toolchains and CI/CD pipelines.

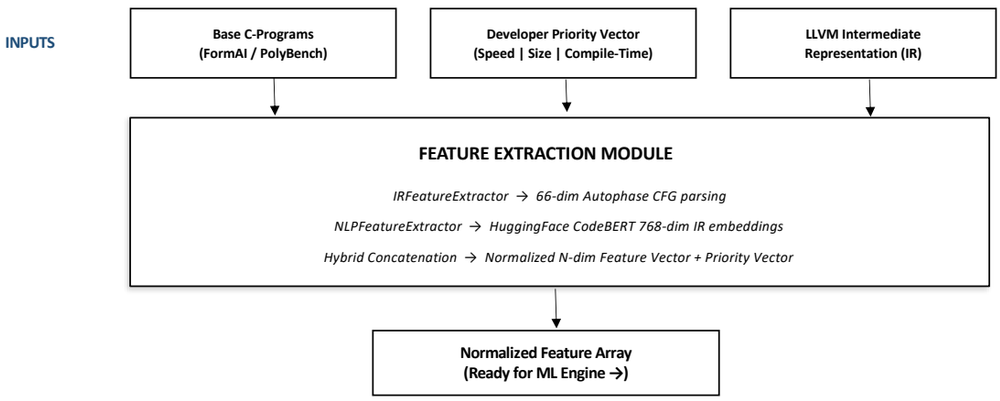
3-A. LITERATURE SURVEY

SNo	Paper Details	Technique Used	Gaps Identified
1	Standard Clang/LLVM -O3 and -O2 heuristics.	Fixed hand-crafted pass sequences applied universally to all C programs.	Rigidity: Performs well on average but fails to extract peak speedups for specialized Matrix / Loop-heavy code.
2	Autophase: Juggling HLS Phase Orderings (Huang et al., 2019).	56-dim Autophase features + Reinforcement Learning for pass ordering.	Feature Limit: Fixed 56-dim features cannot capture complex semantic structures; no developer preference control.

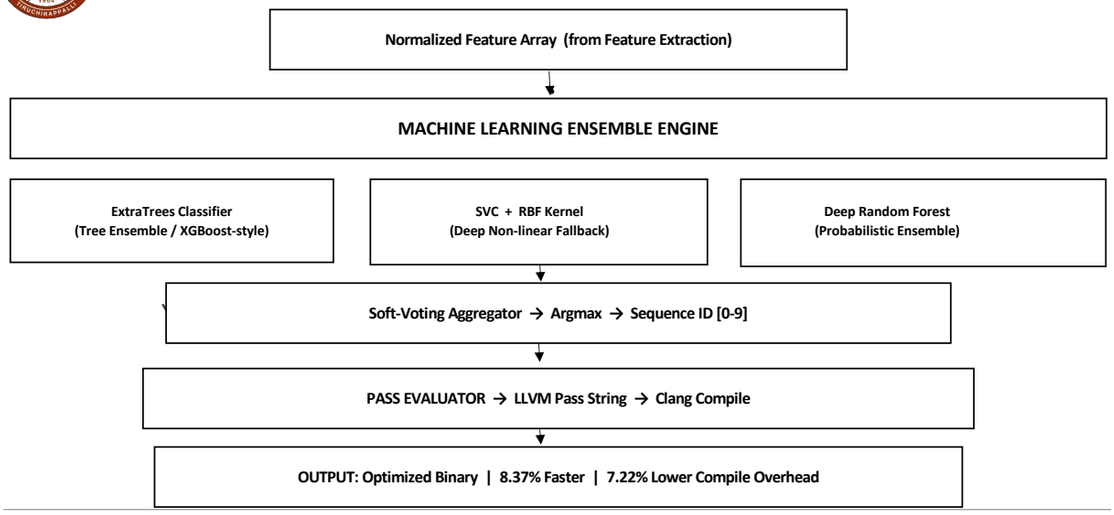
3-B. LITERATURE SURVEY CONTD.

SNo	Paper Details	Technique Used	Gaps Identified
3	Iterative Compilation Search (Genetic Algorithms over LLVM passes).	Evolutionary search over pass orderings — hundreds of compilations per file.	Massive Overhead: Minutes/hours per file. Cannot be used in real-world toolchains.
4	Meta LLM Compiler: Foundation Models of Compiler Optimization (ArXiv, 2024).	7B-parameter LLM trained on LLVM IR for pass selection.	No Priority Tuning: Entirely optimizes for speed; developers cannot steer towards size or compile-time goals.

4-A. BLOCK SCHEMATIC



4-B. BLOCK SCHEMATIC CONTD.



5-A. ALGORITHM

ALGORITHM: PROGRAM CHARACTERISTIC EXTRACTION (PHASE - I):

- **INPUT:**
  - Raw C Source Code files requested by developer.

PROCESS:

- Step 1: Lower C-Program into LLVM Intermediate Representation.
- Step 2: Initialize AutophaseFeatureExtractor.
- Step 3: Parse CFG (Control Flow Graph) extracting 56 node/edge features.
- Step 4: Parse mathematical instructions (Float Ops, Int Ops, Vector Ops).
- Step 5: Augment extracted list with Developer's Priority Vector [Speed, Size, CompileTime].
- Step 6: Normalize features using globally tracked Min-Max scaling.

OUTPUT:

- N-dimensional feature array (69-dim baseline) ready for ML inference.

5-B. ALGORITHM CONTD.

ALGORITHM: ENSEMBLE MODEL INFERENCE (PHASE - II):

- **INPUT:**
  - Normalized IR Feature Array; trained weights for RF, ExtraTrees, SVC.

PROCESS:

- Step 1: Load input array into individual decoupled models concurrently.
- Step 2: Random Forest computes probability tree distribution.
- Step 3: ExtraTrees analyzes generalized boosting distribution.
- Step 4: SVC calculates non-linear RBF decision boundaries.
- Step 5: Soft-voting aggregator computes global weighted mean probability.
- Step 6: Argmax outputs the predicted Sequence ID [0-9].

OUTPUT:

- Optimized target Sequence ID (e.g., Class 0 → aggressive_speed).

5-C. ALGORITHM CONTD.

ALGORITHM: FAST ITERATIVE DATASET GENERATION (PHASE - III):

- **INPUT:**
  - Unlabelled FormAI / Kaggle C-Program repos.

PROCESS:

- Step 1: Extract features for base C-programs.
- Step 2: Generate N randomized Metric Priority permutations.
- Step 3: Iteratively compile program against all 10 Pass Classes.
- Step 4: Measure execution, size, and compile ratios per class.
- Step 5: Compute reward = speed_w×speedup + size_w×reduction + time_w×compile_gain.
- Step 6: Label dataset row with the highest-reward Sequence ID.

OUTPUT:

- Augmented supervised dataset spanning thousands of priority-labelled edge cases.

5-D. ALGORITHM CONTD.

ALGORITHM: PREFERENCE-AWARE SCHEDULING (PHASE - IV):

- **INPUT:**
  - Sequence ID from ML Prediction; LLVM Pass Manager constraints.

PROCESS:

- Step 1: Look up Sequence ID in PassManager dictionary.
- Step 2: Construct string of exact compiler passes (e.g. -inline -loop-unroll).
- Step 3: Subprocess Clang combining backend flags with custom ML passes.
- Step 4: Record output binary footprint and compilation time.
- Step 5: Execute binary natively and monitor OS-level execution duration.

OUTPUT:

- Final Executable with Performance Analytics versus -O3 baseline.

6. EXPERIMENTS & PERFORMANCE METRICS

- **Experiments to be Conducted:**
  - Training on 1,350 augmented C programs from FormAI Dataset & PolyBench benchmarks.
  - 5-Fold Cross Validation on Sklearn SVC + ExtraTrees + Random Forest ensemble.
  - Real-world head-to-head demo: ML-predicted sequence vs Clang -O3 on Matrix Multiplication.
  - Wilcoxon Signed-Rank Test + Cohen's d Effect Size for statistical significance.

Performance Metrics:

- Prediction Accuracy: SVC | Ensemble | Base KNN | Base DT
- Execution Speedup vs –O3
- Compile Overhead Reduction vs –O3
- ML Inference Time
- Adaptability Score

7. IMPLEMENTATION ENVIRONMENT

- **Hardware Ecosystem:**
  - Host: Apple Silicon / Intel architecture / Linux .
  - Memory: 16 GB unified RAM.

Software & Libraries:

- Compiler Infrastructure: LLVM framework (opt / clang).
- Pipeline Language: Python 3.10+.
- Machine Learning: scikit-learn (SVC / ExtraTrees / Random Forest) — Sklearn fallbacks override PyTorch OpenMP crashes on Apple Silicon.
- NLP Representation: HuggingFace Transformers (CodeBERT / CodeT5, inspired by Meta LLM Compiler 2024).
- Data Processing: NumPy, Pandas, Subprocess isolated workers.

8. EXPECTED MILESTONES / WORK PLAN

Milestone	Period	Work Completed / To Be Done
MS0	19.Feb.26 – 05.Apr.26	Literature survey; LLVM toolchain setup; Autophase 66-dim IRFeatureExtractor; CodeBERT NLPFeatureExtractor; Sklearn SVC/ExtraTrees/RF ensemble training; Priority Vector integration; Matrix Multiplication benchmark.
MS1	09.Apr.26 – 05.May.26	Extend dataset to 5,000+ programs from full FormAI corpus; tune ensemble hyperparameters; validate adaptability score across more priority vectors.
MS2	09.May.26 – 19.May.26	Integrate ProGraML graph-based CFG embeddings; compare GNN vs current IRFeatureExtractor; statistical analysis across full PolyBench suite.
MS3	23.May.26 – 31.May.26	Final performance graphs; thesis compilation; Clang C++ pass plugin prototype (future work).

9. REFERENCES

[1] Meta AI Research Team, "Meta Large Language Model Compiler: Foundation Models of Compiler Optimization," ArXiv preprint, 2024.
[2] A. Jain et al., "Machine Learning based prediction of custom optimization passes," PACT / IEEE, 2023.
[3] C. Cummins et al., "ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations," ICML / IEEE, 2021.
[4] S. Venkatakeerthy et al., "IR2Vec: LLVM IR based Scalable Program Embeddings," ACM Transactions on Architecture and Code Optimization (IEEE-cited), 2020.
[5] C. Cummins et al., "CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research," CGO, 2022.
[6] Q. Huang et al., "Autophase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning," MLSys, 2019.
[7] Z. Wang and M. O'Boyle, "Machine Learning in Compiler Optimization," Proceedings of the IEEE, vol. 106, no. 11, 2018.
[8] A. H. Ashouri et al., "Machine Learning in Compilers: Past, Present and Future," HiPEAC, 2018.

THANK YOU

Any Questions?